

# Ключник

Бот-привратник для закрытого чата: заявки с анкетой, оплата в Telegram Stars, автопродление и автокик, сервер 24/7

Закрытый курс сообщества. Файл не пересылаем за пределы чата.

## Содержание

- Урок 0. Что соберём и правила игры
- Урок 1. Скелет и рекламные метки
- Урок 2. Заявки в чат и кнопка «Я человек»
- Урок 3. Анкета и решение админа
- Урок 4. База: кто и откуда пришёл
- Урок 5. Оплата в Telegram Stars и подписка
- Урок 6. Надёжность: ошибки, антифлуд, тесты
- Урок 7. Сервер 24/7
- Урок 8. Финал: пропуск у Привратника
- Приложение А. Вайб-кодинг ботов: промпт и чек-лист
- Приложение Б. Частые ошибки и как их лечить
- Приложение В. Полный код bot.py

# Урок 0. Что соберём и правила игры

---

За 8 уроков ты соберёшь бота, который сторожит закрытый чат. Такого бота можно сразу поставить в своё сообщество или сделать на заказ: это одна из самых частых задач на фрилансе.

## Что будет уметь бот

- Принимать заявки в чат: отсеивать спам-ботов кнопкой «Я человек», задавать анкету и присылать её тебе с кнопками «Принять» и «Отклонить».
- Продавать доступ за Telegram Stars, выдавать одноразовую ссылку, напоминать о продлении и сам удалять тех, у кого кончилась подписка.
- Считать, откуда приходят люди: у каждой рекламы своя ссылка-метка.
- Присылать тебе в личку каждую ошибку с трейсбеком.
- Жить на сервере 24/7 и подниматься после падений.

## Для кого

В каждом уроке два блока советов. «Вайб-кодеру» — если ты пишешь код с ИИ: как ставить задачу и что проверять в ответе. «Кодеру» — практика из боевых ботов, которой нет в документации. В конце — шаблон промпта, чек-лист проверки кода от ИИ и разбор частых ошибок.

## Правила игры

- В каждом уроке с 1 по 7 спрятан ключ. Он появляется, только когда ты делаешь практику: бот что-то ответит, код что-то напечатает, сервер выдаст ошибку.
- Записывай ключи в файл `keys.txt` по порядку и точно как их выдали: с тем же регистром, без пробелов и кавычек.
- В финале твой бот посчитает пропуск из семи ключей, твоего Telegram ID и слова от Привратника — бота нашего сообщества. Ссылка на Привратника — в закреплённом посте.
- Пропуск личный. Чужой тебе не подойдёт, твой не подойдёт другим. Один бот проводит через дверь одного человека.
- Ключи в чате не выкладываем. Помогать друг другу с кодом — можно и нужно.

## Что нужно

- Python 3.10 или новее и любой редактор (подойдёт VS Code).
- Базовый Python: переменные, функции, `if`. Остальное объясним по ходу.
- Второй аккаунт Telegram или друг из сообщества — чтобы проверить заявки.
- К уроку 7 — сервер на Linux. Хватит самого дешёвого VPS.

# Урок 1. Скелет и рекламные метки

---

Сегодня заводим бота и учим его понимать, по какой ссылке пришёл человек. Это первая полезная фишка: дал блогеру одну ссылку, в рекламу — другую, и видишь, что работает.

## Шаг 1. Бот у BotFather

1. Открой @BotFather и отправь /newbot.
2. Придумай имя и username. Username должен заканчиваться на bot.
3. BotFather пришлёт токен вида `7395021846:ААН...`. Это пароль от бота. Число до двоеточия — ID бота, оно пригодится в финале.

## Шаг 2. Проект

```
mkdir kluchnik
cd kluchnik
python -m venv venv
```

Активируй окружение. Windows: `venv\Scripts\activate`. macOS и Linux: `source venv/bin/activate`. Затем:

```
pip install aiogram python-dotenv
```

Рядом с кодом создай файл `.env`:

```
BOT_TOKEN=сюда_твой_токен
OWNER_ID=0
```

## Шаг 3. Код

Файл `bot.py`:

```
import asyncio
import logging
import os

from aiogram import Bot, Dispatcher, F, Router
from aiogram.filters import Command, CommandObject, CommandStart
from aiogram.types import Message
from aiogram.utils.deep_linking import create_start_link
from dotenv import load_dotenv

load_dotenv()
OWNER_ID = int(os.getenv("OWNER_ID", "0"))
router = Router()

@router.message(CommandStart(deep_link=True, deep_link_encoded=True))
async def start_from_link(message: Message, command: CommandObject):
```

```

    await message.answer(f"Привет! Ты пришёл по метке: {command.args}")

@router.message(CommandStart())
async def start(message: Message):
    await message.answer("Привет! Я Ключник, бот закрытого чата.")

@router.message(Command("id"))
async def my_id(message: Message):
    await message.answer(f"Твой ID: {message.from_user.id}")

@router.message(Command("link"), F.from_user.id == OWNER_ID)
async def make_link(message: Message, command: CommandObject, bot: Bot):
    if not command.args:
        await message.answer("Формат: /link метка, например /link youtube")
        return
    await message.answer(await create_start_link(bot, command.args, encode=True))

@router.message(F.chat.type == "private")
async def fallback(message: Message):
    await message.answer("Не понял. Напиши /start")

async def main():
    logging.basicConfig(level=logging.INFO)
    bot = Bot(os.environ["BOT_TOKEN"])
    dp = Dispatcher()
    dp.include_router(router)
    await dp.start_polling(bot)

if __name__ == "__main__":
    asyncio.run(main())

```

Запусти: `python bot.py`. Напиши боту `/id`, впиши число в `.env` вместо нуля в `OWNER_ID` и перезапусти бота (Ctrl+C и снова `python bot.py`). Теперь отправь `/link youtube` — бот пришлёт ссылку с меткой.

## Как это работает

- Бот раз за разом спрашивает Telegram «есть новое?». Это `polling`. Каждое событие — сообщение, нажатие кнопки, заявка в чат — приходит как `update`.
- `Dispatcher` раздаёт `update` роутерам, `Router` — коробка с хендлерами. Хендлер — функция с декоратором, который говорит, на что она реагирует.
- Хендлеры проверяются сверху вниз, событие забирает первый подходящий. Поэтому `fallback`, который ловит всё подряд, всегда стоит последним.
- `F.from_user.id == OWNER_ID` — фильтр: `/link` работает только у тебя. У остальных команда провалится в `fallback`.
- Ссылка `t.me/бот?start=метка` отправляет боту «`/start` метка». В метке можно только латиницу, цифры, `_` и `-`, до 64 символов. `encode=True` кодирует метку в `base64`, поэтому в неё

влезают любые символы, хоть кириллица. `deep_link_encoded=True` декодирует её обратно.

**Ключ 1.** Открой ссылку `https://t.me/ИМЯ_ТВОЕГО_БОТА?start=c3Jj0nZvcn90YS03` (username бота без @) и нажми «Запустить». Метка из ответа бота — ключ 1. Запиши её в `keys.txt`.

**Вайб-кодеру.** Первой строкой любого запроса к ИИ пиши версии: «Python 3.12, aiogram 3.x, не 2.x». Код для aiogram 2 узнаётся сразу: `executor.start_polling, @dp.message_handler`. Увидел такое — проси переписать под aiogram 3.

**Кодеру.** Токен и ID живут только в `.env`. Если токен хоть раз попал в git, не переписывай историю — сразу отправь BotFather /revoke: копии репозитория уже могли разойтись.

## Урок 2. Заявки в чат и кнопка «Я человек»

---

Сегодня бот начнёт встречать каждого, кто подал заявку в твой чат. Спам-боты кнопки не нажимают, поэтому одна кнопка «Я человек» отсекает большую часть спама.

### Шаг 1. Тестовый чат

1. Создай группу. В настройках: «Пригласительные ссылки» → «Создать ссылку» → включи «Заявки на вступление».
2. Добавь бота в администраторы с правом приглашать пользователей. Без этого права бот не увидит заявок.
3. Отправь в группе /chatid. Бот ответит ID вида -100... — впиши его в .env строкой  
`CHAT_ID=-100...`

### Шаг 2. Код

Добавь в импорты:

```
from aiogram.exceptions import TelegramBadRequest
from aiogram.filters.callback_data import CallbackData
from aiogram.types import CallbackQuery, ChatJoinRequest, Message
from aiogram.utils.keyboard import InlineKeyboardBuilder
```

Под строкой с OWNER\_ID:

```
CHAT_ID = int(os.getenv("CHAT_ID", "0"))
```

Под строкой `router = Router()`:

```
class Gate(CallbackData, prefix="jr"):
    act: str
    user_id: int
```

Хендлеры — выше fallback:

```
@router.message(Command("chatid"))
async def chat_id(message: Message):
    await message.answer(f"ID этого чата: {message.chat.id}")

@router.chat_join_request(F.chat.id == CHAT_ID)
async def on_join_request(request: ChatJoinRequest, bot: Bot):
    kb = InlineKeyboardBuilder()
    kb.button(text="Я человек", callback_data=Gate(act="human", user_id=request.from_user.id))
    await bot.send_message(
        request.user_chat_id,
        f"Ты подал заявку в «{request.chat.title}». Нажми кнопку, чтобы продолжить.",
        reply_markup=kb.as_markup(),
    )
```

```
@router.callback_query(Gate.filter(F.act == "human"))
async def human(callback: CallbackQuery, bot: Bot):
    try:
        await bot.approve_chat_join_request(CHAT_ID, callback.from_user.id)
    except TelegramBadRequest:
        await callback.answer("Заявка уже обработана или устарела.", show_alert=True)
    return
    await callback.message.edit_text("Готово, добро пожаловать в чат!")
    await callback.answer()
```

Перезапусти бота и попроси друга (или свой второй аккаунт) подать заявку по ссылке. Человек получит сообщение с кнопкой, нажмёт — и окажется в чате.

## Как это работает

- Заявка приходит отдельным типом события: `chat_join_request`. Писать человеку можно через `request.user_chat_id`, даже если он ни разу не запускал бота. Окно короткое — 5 минут, поэтому пишем сразу.
- Inline-кнопка отправляет боту невидимую строку `callback_data` — до 64 байт. Класс `Gate` сам собирает её из полей и разбирает обратно: в хендлере `user_id` уже число.
- `Gate.filter(F.act == "human")` пропускает только кнопки «Я человек».
- `callback.answer()` обязателен. Без него у человека крутятся часики на кнопке.

**Ключ 2.** Открой второй терминал в папке проекта, активируй `venv` и выполни: `python -c "from bot import Gate; print(Gate(act='human', user_id=12).pack())"`. Напечатанная строка — ключ 2.

Бот при этом не запустится: `main()` вызывается, только когда файл запускают напрямую. Для этого и нужна строчка `if __name__ == "__main__":`.

**Вайб-кодеру.** ИИ часто забывает `callback.answer()`. Симптом — часики на кнопке 10–15 секунд. Нашёл `callback`-хендлер без `answer` — допиши сам.

**Кодеру.** Права проверяй в хендлере, а не тем, кому показал кнопку. В следующем уроке кнопки админа защищены фильтром по `OWNER_ID`: даже если кнопка попадёт к другому человеку, она ничего не сделает.

## Урок 3. Анкета и решение админа

---

Сегодня кнопка «Я человек» станет началом анкеты. Бот задаст два вопроса, пришлёт тебе карточку заявки, а ты решишь, пускать ли человека.

### Зачем состояния

Бот получает сообщения по одному и сам не помнит, о чём шла речь. Пришло «Пишу ботов на заказ» — это ответ на анкету или просто текст? Состояние (FSM, машина состояний) — пометка «этот человек сейчас отвечает на такой-то вопрос».

Добавь в импорты:

```
from aiogram.fsm.context import FSMContext
from aiogram.fsm.state import State, StatesGroup
```

Рядом с классом Gate:

```
class Apply(StatesGroup):
    waiting_who = State()
    waiting_from = State()
```

Что умеет `state` в хендлере: `set_state` ставит пометку, `update_data` и `get_data` хранят ответы, `clear` всё стирает. Хендлер с фильтром `Apply.waiting_who` сработает только у того, кто стоит в этом состоянии.

### Код

Замени хендлер `human` — теперь он не пускает сразу, а начинает анкету:

```
@router.callback_query(Gate.filter(F.act == "human"))
async def human(callback: CallbackQuery, state: FSMContext):
    await state.set_state(Apply.waiting_who)
    await callback.message.edit_text("Спасибо! Два вопроса.\n1. Кто ты и чем занимаешься?")
    await callback.answer()
```

Новые хендлеры. `/cancel` ставь выше остальных, всё вместе — выше `fallback`:

```
@router.message(Command("cancel"))
async def cancel(message: Message, state: FSMContext):
    await state.clear()
    await message.answer("Отменил.")

@router.callback_query(Gate.filter(F.act.in_({"ok", "no"})), F.from_user.id == OWNER_ID)
async def decide(callback: CallbackQuery, callback_data: Gate, bot: Bot):
    uid, ok = callback_data.user_id, callback_data.act == "ok"
    try:
        if ok:
            await bot.approve_chat_join_request(CHAT_ID, uid)
```

```

        else:
            await bot.decline_chat_join_request(CHAT_ID, uid)
    except TelegramBadRequest as e:
        await callback.answer(f"Не вышло: {e.message}"[:200], show_alert=True)
        return
    with suppress(TelegramAPIError):
        await bot.send_message(uid, "Заявка принята. Добро пожаловать!" if ok else "Заявка отклонена.")
        verdict = "принята" if ok else "отклонена"
        await callback.message.edit_text(f"{callback.message.text}\n\nЗаявка {verdict}.")
        await callback.answer()

@router.message(Apply.waiting_who, F.text)
async def apply_who(message: Message, state: FSMContext):
    await state.update_data(who=message.text)
    await state.set_state(Apply.waiting_from)
    print(await state.get_state())
    await message.answer("2. Откуда узнал о нас?")

@router.message(Apply.waiting_from, F.text)
async def apply_from(message: Message, state: FSMContext, bot: Bot):
    data = await state.get_data()
    await state.clear()
    user = message.from_user
    nick = f" @{user.username}" if user.username else ""
    kb = InlineKeyboardBuilder()
    kb.button(text="Принять", callback_data=Gate(act="ok", user_id=user.id))
    kb.button(text="Отклонить", callback_data=Gate(act="no", user_id=user.id))
    await bot.send_message(
        OWNER_ID,
        f"Заявка: {user.full_name}{nick}, id {user.id}\n\n"
        f"Кто: {data['who']}\nОткуда: {message.text}",
        reply_markup=kb.as_markup(),
    )
    await message.answer("Анкета у админа. Ответ придёт сюда.")

```

Для `suppress` и `TelegramAPIError` добавь в импорты `from contextlib import suppress` и допиши `TelegramAPIError` в строку с `TelegramBadRequest`.

## Ловушки порядка

- В aiogram 3 хендлер без фильтра состояния срабатывает в любом состоянии. Окажись fallback выше анкеты — он перехватит все ответы.
- Окажись `/cancel` ниже анкеты — текст `«/cancel»` запишется как ответ, и сбежать не получится.
- Состояния по умолчанию живут в памяти процесса. Перезапустил бота — анкеты в процессе потеряны. В больших ботах состояния хранят в Redis.

**Ключ 3.** Поддай заявку со второго аккаунта, нажми «Я человек» и ответь на первый вопрос. Посмотри в консоль, где запущен бот: строка, которую напечатал `print(await state.get_state())`, — ключ 3. После этого `print` можно удалить.

**Вайб-кодеру.** Если ИИ хранит ответы в глобальном словаре вроде `answers = {}` — проси переделать на FSM. Глобальный словарь путает людей, которые заполняют анкету одновременно.

**Кодеру.** Текст от пользователя отправляй без `parse_mode` или пропускай через `html.escape()`. Иначе первый же человек, который напишет «<3», уронит отправку ошибкой «can't parse entities». Наш бот шлёт анкеты простым текстом.

## Урок 4. База: кто и откуда пришёл

---

Сегодня бот начнёт записывать каждого нового человека и его метку. Команда /admin покажет, какая реклама приводит людей. Данные переживут перезапуск.

SQLite — база данных в одном файле: сервер не нужен, модуль уже встроен в Python. Для бота берём `aiosqlite` — ту же SQLite, но с `await`, чтобы запрос к диску не замораживал бота для остальных.

```
pip install aiosqlite
```

Добавь `import aiosqlite` в импорты, а под `CHAT_ID` — строку `DB = "kluchnik.db"`. Функции базы — под `router = Router()`:

```
async def init_db():
    async with aiosqlite.connect(DB) as db:
        await db.executescript(
            """
            CREATE TABLE IF NOT EXISTS users (
                user_id INTEGER PRIMARY KEY,
                source TEXT,
                joined TEXT DEFAULT CURRENT_TIMESTAMP
            );
            """
        )
        await db.commit()

async def save_user(user_id: int, source: str | None):
    async with aiosqlite.connect(DB) as db:
        await db.execute(
            "INSERT OR IGNORE INTO users (user_id, source) VALUES (?, ?)",
            (user_id, source),
        )
        await db.commit()

async def sources_report():
    async with aiosqlite.connect(DB) as db:
        cur = await db.execute(
            "SELECT COALESCE(source, 'без метки'), COUNT(*) AS c FROM users "
            "GROUP BY source ORDER BY c DESC"
        )
        return await cur.fetchall()
```

Три правки в старом коде:

1. В `main()` перед `await dp.start_polling(bot)` добавь `await init_db()`.
2. В `start_from_link` первой строкой добавь `await save_user(message.from_user.id, command.args)`, а в `start` — `await save_user(message.from_user.id, None)`.
3. Новый хендлер выше `fallback`:

```
@router.message(Command("admin"), F.from_user.id == OWNER_ID)
async def admin(message: Message):
    rows = await sources_report()
    lines = [f"{source}: {count}" for source, count in rows] or ["пока никого"]
    await message.answer("Откуда пришли:\n" + "\n".join(lines))
```

## Как это работает

- `?` в запросе — место для значения. Значения передаём отдельно кортежем. Данные пользователя в SQL через f-строку не вставляем никогда: это SQL-инъекция.
- `INSERT OR IGNORE` вместе с `PRIMARY KEY` записывает человека один раз. Пришёл сначала с YouTube, потом из рекламы — засчитается YouTube. Это модель «первого касания»: заслуга достаётся каналу, который привёл человека первым.
- `COALESCE(source, 'без метки')` подставляет текст вместо пустой метки.
- `GROUP BY source` собирает людей с одной меткой в группу, `COUNT(*)` считает размер группы, `ORDER BY` с `DESC` ставит самую большую первой.

**Ключ 4.** Сначала предскажи ответ в уме — помни про первое касание. Потом создай файл `key4.py` с кодом ниже и запусти. Напечатанная строка — ключ 4.

```
import sqlite3

db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE users (user_id INTEGER PRIMARY KEY, source TEXT)")
db.executemany(
    "INSERT OR IGNORE INTO users VALUES (?, ?)",
    [(1, "yt"), (2, "ads"), (1, "ads"), (3, "yt"), (4, None), (2, "yt"), (5, "ads"), (6, "ads")],
)
source, count = db.execute(
    "SELECT COALESCE(source, 'none'), COUNT(*) AS c FROM users "
    "GROUP BY source ORDER BY c DESC, source LIMIT 1"
).fetchone()
print(f"{source}:{count}")
```

**Вайб-кодеру.** Поищи в коде от ИИ SQL с f-строкой вроде `f"... WHERE id = {user_id}"`. Это дыра. Правильно — только `?` и кортеж значений.

**Кодеру.** Время в базе храни в UTC и в формате `гггг-мм-дд чч:мм:сс` — так SQLite сравнивает даты обычным сравнением строк. Часовой пояс пользователя подставляй только при выводе. SQLite спокойно тянет бота на десятки тысяч человек; PostgreSQL нужен, когда в базу пишут несколько процессов сразу.

## Урок 5. Оплата в Telegram Stars и подписка

---

Сегодня бот начнёт продавать доступ в чат на 30 дней. После оплаты он выдаст одноразовую ссылку, за 3 дня до конца напомнит о продлении, а не продлившего сам удалит из чата.

### Почему Stars

За цифровые товары и услуги внутри Telegram бот берёт оплату в Telegram Stars — это правило платформы. Плюс для нас: не нужен ни платёжный провайдер, ни договор, ни `provider_token`. Валюта Stars в коде — `XTR`, сумма — целое число звёзд.

Добавь в импорты `from datetime import timedelta`, а в строку типов — `LabeledPrice` и `PreCheckoutQuery`. Под строкой с DB:

```
PRICE_STARS = 100
SUB_DAYS = 30
```

### База подписок

В `init_db` внутри `executescript` после таблицы `users` допиши вторую таблицу:

```
CREATE TABLE IF NOT EXISTS subs (
    user_id INTEGER PRIMARY KEY,
    until TEXT NOT NULL,
    reminded INTEGER DEFAULT 0
);
```

Функции — к остальным функциям базы:

```
async def extend_sub(user_id: int, days: int) -> str:
    period = f"{days} days"
    async with aiosqlite.connect(DB) as db:
        await db.execute(
            "INSERT INTO subs (user_id, until) VALUES (?, datetime('now', ?)) "
            "ON CONFLICT(user_id) DO UPDATE SET "
            "until = datetime(max(until, datetime('now'))), ?), reminded = 0",
            (user_id, period, period),
        )
        await db.commit()
    cur = await db.execute("SELECT until FROM subs WHERE user_id = ?", (user_id,))
    return (await cur.fetchone())[0]

async def sub_until(user_id: int) -> str | None:
    async with aiosqlite.connect(DB) as db:
        cur = await db.execute(
            "SELECT until FROM subs WHERE user_id = ? AND until > datetime('now')",
            (user_id,),
        )
        row = await cur.fetchone()
        return row[0] if row else None
```

`extend_sub` делает «upsert»: нет подписки — создаёт, есть — продлевает. `max(until, datetime('now'))` решает важный вопрос: продлил заранее — дни прибавятся к концу подписки, продлил после окончания — отсчёт пойдёт от сегодня.

## Покупка

Хендлеры — выше fallback:

```
@router.message(Command("buy"))
async def buy(message: Message):
    await message.answer_invoice(
        title="Доступ в закрытый чат",
        description=f"{SUB_DAYS} дней доступа",
        payload=f"sub:{SUB_DAYS}",
        currency="XTR",
        prices=[LabeledPrice(label="Доступ", amount=PRICE_STARS)],
    )

@router.message(Command("status"))
async def status(message: Message):
    until = await sub_until(message.from_user.id)
    await message.answer(f"Доступ до {until} UTC." if until else "Доступа нет. Купить: /buy")

@router.message(Command("refund"), F.from_user.id == OWNER_ID)
async def refund(message: Message, command: CommandObject, bot: Bot):
    parts = (command.args or "").split()
    if len(parts) != 2 or not parts[0].isdigit():
        await message.answer("Формат: /refund USER_ID CHARGE_ID")
        return
    await bot.refund_star_payment(int(parts[0]), parts[1])
    await message.answer("Звёзды возвращены.")

@router.pre_checkout_query()
async def pre_checkout(query: PreCheckoutQuery):
    await query.answer(ok=True)

@router.message(F.successful_payment)
async def paid(message: Message, bot: Bot):
    pay = message.successful_payment
    until = await extend_sub(message.from_user.id, SUB_DAYS)
    link = await bot.create_chat_invite_link(CHAT_ID, member_limit=1,
    expire_date=timedelta(days=1))
    await message.answer(f"Оплата прошла. Доступ до {until} UTC.\nТвоя одноразовая ссылка:
    {link.invite_link}")
    await bot.send_message(
        OWNER_ID,
        f"+{pay.total_amount} звёзд от {message.from_user.id}\n"
        f"Возврат: /refund {message.from_user.id} {pay.telegram_payment_charge_id}",
    )
```

Как проходит оплата: `/buy` присылает счёт → человек жмёт «Оплатить» → Telegram спрашивает бота через `pre_checkout_query`, можно ли продавать (на ответ 10 секунд) → списывает звёзды →

присылает сообщение `successful_payment`. Доступ выдаём только по нему.

Ссылка с `member_limit=1` пускает ровно одного человека и сгорает через сутки — переслать её другу бесполезно. Такие ссылки работают и в чате с заявками: по ним человек заходит сразу, без анкеты.

Обнови тексты: в start напиши «/buy — купить доступ, /status — мой доступ», в fallback — «Не понял. Команды: /buy, /status».

## Сторож подписок

Фоновая задача раз в 10 минут проверяет базу. Функции — к остальным функциям базы:

```
async def check_subs(bot: Bot):
    async with aisqlite.connect(DB) as db:
        cur = await db.execute(
            "SELECT user_id, until FROM subs WHERE reminded = 0 "
            "AND until > datetime('now') AND until <= datetime('now', '+3 days')"
        )
        for user_id, until in await cur.fetchall():
            with suppress(TelegramAPIError):
                await bot.send_message(user_id, f"Доступ заканчивается {until} UTC. Продлить: /buy")
            await db.execute("UPDATE subs SET reminded = 1 WHERE user_id = ?", (user_id,))

        cur = await db.execute("SELECT user_id FROM subs WHERE until <= datetime('now')")
        for (user_id,) in await cur.fetchall():
            try:
                await bot.ban_chat_member(CHAT_ID, user_id)
                await bot.unban_chat_member(CHAT_ID, user_id, only_if_banned=True)
            except TelegramBadRequest as e:
                logging.warning("Не смог удалить %s: %s", user_id, e)
            await db.execute("DELETE FROM subs WHERE user_id = ?", (user_id,))
        await db.commit()

async def watch_subs(bot: Bot):
    while True:
        try:
            await check_subs(bot)
        except Exception:
            logging.exception("Проверка подписок упала")
        await asyncio.sleep(600)
```

В main() замени строку `await dp.start_polling(bot)` на три:

```
watcher = asyncio.create_task(watch_subs(bot)) # держим ссылку, иначе задачу съест сборщик мусора
await dp.start_polling(bot)
watcher.cancel()
```

- Пара «бан + разбан» — штатный способ удалить человека из чата без вечного бана. Он сможет вернуться, когда снова оплатит.

- Флаг `reminded` не даёт слать напоминание каждые 10 минут. При продлении он сбрасывается.

Проверка без лишних трат: поставь `PRICE_STARS = 1`, купи доступ у своего бота, а потом нажми команду `/refund` из уведомления — звезда вернётся. Звёзды для теста покупаются в самом Telegram, хватит самого маленького пакета.

**Ключ 5.** Это тот же запрос, по которому сторож ищет, кому напомнить. Предскажи ответ, потом запусти `key5.py`. Напечатанная строка — ключ 5.

```
import sqlite3

db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE subs (user_id INTEGER PRIMARY KEY, until TEXT)")
db.executemany("INSERT INTO subs VALUES (?, ?)", [
    (101, "2026-10-02 10:00:00"), (102, "2026-09-30 23:00:00"),
    (103, "2026-10-04 09:00:00"), (104, "2026-10-03 18:00:00"),
    (105, "2026-11-01 00:00:00"),
])
now = "2026-10-01 12:00:00"
rows = db.execute(
    "SELECT user_id FROM subs WHERE until > ? AND until <= datetime(?, '+3 days') "
    "ORDER BY until",
    (now, now),
).fetchall()
print("-".join(str(r[0]) for r in rows))
```

**Вайб-кодеру.** ИИ по старой памяти пишет `provider_token` и валюту `RUB`. Для цифровых товаров в боте — только Stars: `currency="XTR"`, `provider_token` не нужен.

**Кодеру.** Доступ выдавай только по `successful_payment`, никогда по `pre_checkout_query`. Сохраняй `telegram_payment_charge_id` — без него не сделать возврат. Фоновую задачу без `try/except` внутри цикла первая же ошибка сети убьёт молча, и подписки перестанут проверяться.

## Урок 6. Надёжность: ошибки, антифлуд, тесты

---

Сегодня бот научится присылать тебе каждую ошибку, перестанет захлёбываться от флуда, а ты научишься проверять хендлеры без Telegram.

### Ошибки — тебе в личку

Добавь `import traceback` и `ErrorEvent` в импорты. Хендлер — выше `fallback`:

```
@router.errors()
async def on_error(event: ErrorEvent, bot: Bot):
    logging.error("Ошибка: %r", event.exception, exc_info=event.exception)
    tb = "".join(traceback.format_exception(event.exception))
    with suppress(TelegramAPIError):
        await bot.send_message(OWNER_ID, f"Ошибка в боте: {event.exception!r}\n\n{tb[-3500:]}")
```

Проверь: отправь `/refund 1 abc`. Telegram ответит ошибкой, и через секунду у тебя в личке будет трейсбек. Теперь ты узнаёшь о проблеме раньше пользователей.

### Что такое middleware

Middleware — код, который оборачивает хендлеры: делает что-то до них и после. Логирование, антифлуд, бан-лист — всё это middleware, чтобы не копировать одно и то же в каждый хендлер. В `aiogram` их два вида:

- `outer` — срабатывает на каждое событие, ещё до проверки фильтров;
- `inner` (просто middleware) — срабатывает, только если нашёлся хендлер, чьи фильтры подошли.

Middleware — это класс с методом `__call__(handler, event, data)`. Вызвал внутри `await handler(event, data)` — событие пошло дальше. Не вызвал — событие остановлено.

### Антифлуд

Добавь `import time` и допиши `BaseMiddleware` в строку `from aiogram import ...`. Под `router = Router()`:

```
class AntiFlood(BaseMiddleware):
    def __init__(self, delay: float = 0.7):
        self.delay = delay
        self.last: dict[int, float] = {}

    async def __call__(self, handler, event, data):
        user = data.get("event_from_user")
        if user:
            now = time.monotonic()
            if now - self.last.get(user.id, 0) < self.delay:
                return # слишком часто: молча пропускаем
            self.last[user.id] = now
        return await handler(event, data)
```

```
router.message.outer_middleware(AntiFlood())
```

`event_from_user` aiogram сам кладёт в `data` — это автор события. `time.monotonic()` — часы, которые не скачут при переводе времени.

**Ключ 6.** Middleware можно проверить без Telegram: aiogram умеет «скормить» диспетчеру поддельное сообщение. Создай файл `mw_test.py` с кодом ниже. До запуска предскажи, что он напечатает, потом запусти. Напечатанное слово — ключ 6.

```
import asyncio
from datetime import datetime

from aiogram import BaseMiddleware, Bot, Dispatcher, F, Router
from aiogram.types import Chat, Message, Update, User

log = []

class Mark(BaseMiddleware):
    def __init__(self, before, after):
        self.before, self.after = before, after

    async def __call__(self, handler, event, data):
        log.append(self.before)
        result = await handler(event, data)
        log.append(self.after)
        return result

router = Router()
router.message.outer_middleware(Mark("g", "e"))
router.message.outer_middleware(Mark("a", "t"))
router.message.middleware(Mark("r", "d"))

@router.message(F.text == "тык")
async def knock(message: Message):
    log.append("o")

def fake(text: str) -> Update:
    return Update(update_id=1, message=Message(
        message_id=1, date=datetime.now(), text=text,
        chat=Chat(id=1, type="private"),
        from_user=User(id=1, is_bot=False, first_name="Test"),
    ))

async def main():
    dp = Dispatcher()
    dp.include_router(router)
    bot = Bot("42:TEST") # фейковый токен: в сеть мы не ходим
    await dp.feed_update(bot, fake("привет"))
    print("".join(log))
```

```
await bot.session.close()

asyncio.run(main())
```

Потом замени «привет» на «тук» и запусти ещё раз. Объясни себе, откуда взялись новые буквы и почему они встали в середину.

## Тесты хендлеров без Telegram

Тот же приём проверяет и настоящего бота. Подменяем сессию — и все вызовы Telegram API попадают в список, а не в сеть. Файл `test_bot.py`:

```
import asyncio
import os
from datetime import datetime

os.environ["OWNER_ID"] = "1"

from aiogram import Bot, Dispatcher
from aiogram.client.session.base import BaseSession
from aiogram.types import Chat, Message, Update, User

import bot as app

class FakeSession(BaseSession):
    def __init__(self):
        super().__init__()
        self.sent = []

    async def make_request(self, bot, method, timeout=None):
        self.sent.append(method)
        return True

    async def close(self): ...
    async def stream_content(self, *args, **kwargs): ...

async def main():
    session = FakeSession()
    bot = Bot("42:TEST", session=session)
    dp = Dispatcher()
    dp.include_router(app.router)
    update = Update(update_id=1, message=Message(
        message_id=1, date=datetime.now(), text="/id",
        chat=Chat(id=5, type="private"),
        from_user=User(id=5, is_bot=False, first_name="Test"),
    ))
    await dp.feed_update(bot, update)
    assert session.sent[-1].text == "Твой ID: 5"
    print("ok")

asyncio.run(main())
```

Такой тест проверяет хендлер за миллисекунды и без интернета. Попроси ИИ дописать тесты на каждую команду — с ними правки перестают ломать то, что уже работало.

**Вайб-кодеру.** Когда что-то падает, отдавай ИИ три вещи: полный трейсбек, код хендлера и версию `aiogram` ( `pip show aiogram` ). «Не работает» без трейсбека — это гадание.

**Кодеру.** Рассылка по базе: не больше 25 сообщений в секунду. Ловишь `TelegramRetryAfter` — спишь `e.retry_after` секунд и повторяешь. Ловишь `TelegramForbiddenError` — человек заблокировал бота, отметь это в базе и больше ему не пиши.

## Урок 7. Сервер 24/7

---

Сегодня переносим бота на сервер, чтобы он работал без твоего компьютера, сам поднимался после падений и делал бэкапы.

### Подготовка проекта

Файл `.gitignore` в папке проекта:

```
venv/  
.env  
*.db  
__pycache__/
```

Список зависимостей: `pip freeze > requirements.txt`. Залей код в приватный репозиторий на GitHub. Файл `.env` туда не попадёт — на сервере создашь его заново.

### Сервер

Подойдёт самый дешёвый VPS с Ubuntu 24.04. Зайди на него по SSH и выполни:

```
sudo apt update  
sudo apt install -y python3-venv git sqlite3  
git clone https://github.com/ТВОЙ_ЛОГИН/kluchnik.git ~/kluchnik  
cd ~/kluchnik  
python3 -m venv venv  
venv/bin/pip install -r requirements.txt  
nano .env
```

Для приватного репозитория GitHub спросит логин и токен доступа (Personal access token). В `.env` впиши те же `BOT_TOKEN`, `OWNER_ID` и `CHAT_ID`, что дома. Проверь запуск руками: `venv/bin/python bot.py`, затем `Ctrl+C`.

### systemd: бот как служба

Узнай своё имя пользователя (`whoami`) и путь к папке (`pwd`). Создай файл службы: `sudo nano /etc/systemd/system/kluchnik.service`.

```
[Unit]  
Description=Kluchnik bot  
After=network-online.target  
Wants=network-online.target  
  
[Service]  
User=ТВОЙ_ПОЛЬЗОВАТЕЛЬ  
WorkingDirectory=/home/ТВОЙ_ПОЛЬЗОВАТЕЛЬ/kluchnik  
ExecStart=/home/ТВОЙ_ПОЛЬЗОВАТЕЛЬ/kluchnik/venv/bin/python bot.py  
Restart=always  
RestartSec=5
```

```
[Install]
WantedBy=multi-user.target
```

Если заходишь под root, путь будет /root/kluchnik, а пользователь — root. Запускаем:

```
sudo systemctl daemon-reload
sudo systemctl enable --now kluchnik
sudo systemctl status kluchnik
journalctl -u kluchnik -f
```

- `Restart=always` : упал бот — systemd поднимет его через 5 секунд.
- `enable` : бот стартует сам после перезагрузки сервера.
- `journalctl -f` показывает живой лог. Выход — Ctrl+C.
- Обновить код: `cd ~/kluchnik && git pull && sudo systemctl restart kluchnik`.

## Бэкап базы каждую ночь

В базе — подписки, за которые люди заплатили. Потерять её нельзя. Открой `crontab -e` и добавь строку:

```
0 4 * * * cd ~/kluchnik && mkdir -p backups && sqlite3 kluchnik.db ".backup
backups/kluchnik-$(date +%F).db"
```

Команда `.backup` безопасна даже при работающем боте, в отличие от простого копирования файла. Знак % в crontab экранируется обратной чертой — без неё строка не работает.

## Один токен — один запущенный бот

Telegram отдаёт обновления только одному процессу. Запустишь второй с тем же токеном — они начнут драться за обновления. Самая частая беда: бот уже работает на сервере, а ты запускаешь его ещё и дома для отладки. Для отладки заведи у BotFather второго, тестового бота.

**Ключ 7.** Пока бот работает на сервере, запусти его же у себя на компьютере: `python bot.py`. Подожди 10–20 секунд и отправь боту пару сообщений. В одной из консолей (дома или на сервере в `journalctl -u kluchnik -f`) появится ошибка `Failed to fetch updates - TelegramConflictError: Telegram server says - ...`. Первое слово после «Telegram server says -», маленькими буквами, — ключ 7. Потом останови домашнего бота.

**Вайб-кодеру.** Не проси ИИ «задеплоить за тебя». Проси объяснить каждую команду, прежде чем её запускать. На сервере ты один на один с консолью, и `rm -rf` из чужого ответа не откатить.

**Кодеру.** Webhook вместо polling нужен, когда событий тысячи в минуту или бот живёт на serverless. Для бота такого размера polling проще: ему не нужны ни домен, ни сертификат, ни открытый порт.

## Урок 8. Финал: пропуск у Привратника

---

Сегодня твой бот сам заработает тебе пропуск в чат выпускников.

### Как проходит экзамен

1. Пишешь Привратнику `/gate`. Он выдаёт одноразовое слово из 8 символов. Слово живёт 15 минут.
2. Отправляешь своему боту `/proof СЛОВО`.
3. Бот считает код из семи ключей, твоего ID, ID бота и слова и отвечает: `PROOF слово КОД`.
4. Пересылаешь этот ответ Привратнику. Именно «Переслать», а не скопировать текст.
5. Привратник проверяет код и выдаёт одноразовую ссылку в чат выпускников.

Почему это не подделать:

- Код зависит от твоего Telegram ID. Ответ чужого бота для тебя не сработает.
- Привратник видит, от какого бота переслано сообщение, и сам знает его ID.
- Один бот проводит через дверь только одного человека.
- Слово одноразовое и быстро протухает.

### Код

Добавь `import hashlib` в импорты. После строки с `SUB_DAYS`:

```
KEYS = ["ключ1", "ключ2", "ключ3", "ключ4", "ключ5", "ключ6", "ключ7"]
```

Подставь свои ключи из `keys.txt` в порядке уроков. Регистр и знаки — точно как их выдали.

Функцию — к остальным функциям, хендлер — выше `fallback`:

```
def make_proof(user_id: int, bot_id: int, word: str) -> str:
    raw = "|".join([*KEYS, str(user_id), str(bot_id), word])
    return hashlib.sha256(raw.encode()).hexdigest()[:10].upper()

@router.message(Command("proof"), F.from_user.id == OWNER_ID)
async def proof(message: Message, command: CommandObject, bot: Bot):
    if not command.args:
        await message.answer("Формат: /proof СЛОВО_ОТ_ПРИВРАТНИКА")
        return
    word = command.args.strip()
    code = make_proof(message.from_user.id, bot.id, word)
    await message.answer(f"PROOF {word} {code}")
```

### Как это работает

- `"|".join(...)` склеивает ключи, твой ID, ID бота и слово через вертикальную черту.
- `sha256` — хеш-функция: из любой строки получается 64 символа. Поменяй на входе один символ — изменится весь хеш. Обратно хеш не разворачивается, поэтому по чужому коду

ключи не узнать.

- `[:10].upper()` — первые 10 символов, заглавными буквами.
- `bot.id` — ID бота, то самое число из токена (урок 1).
- `F.from_user.id == OWNER_ID` — пропуск выдаётся только хозяину бота.

Тот же приём — подпись данных хешем с секретом — защищает ссылки на оплату, вебхуки и данные мини-приложений Telegram. Ты только что собрал его руками.

Обнови бота на сервере (урок 7) и иди к Привратнику.

## Если Привратник не пускает

- «Код не сходится» — проверь ключи: порядок, регистр, лишние пробелы и кавычки. Какой ключ неверный, Привратник не скажет.
- «Слово протухло» — возьми новое: `/gate`.
- «Это копия» — нужно переслать сообщение, а не копировать его текст.
- «Этот бот уже провёл другого человека» — нужен свой бот.
- Бот отвечает «Не понял» — хендлер `/proof` стоит ниже `fallback` или в `.env` записан не твой `OWNER_ID`.

# Приложение А. Вайб-кодинг ботов: промпт и чек-лист

---

## Шаблон промпта

Копируй, заполняй скобки, отправляй. Чем точнее задача и пример диалога, тем меньше переделок.

Ты опытный разработчик Telegram-ботов.  
Стек: Python 3.12, aiogram 3.x (не 2.x), aioredis, python-dotenv.  
Вот мой bot.py целиком:  
<вставь код>

Задача: <что должно появиться>

Пример диалога:

<пользователь пишет...> -> <бот отвечает...>

Требования:

- токены и ID только из .env;
- SQL только с плейсхолдерами ?;
- callback.answer() в каждом callback-хендлере;
- новые хендлеры ставь выше fallback и скажи, куда именно вставить;
- не меняй то, о чём я не просил.

Ответ: только изменённые куски кода, место вставки и как проверить руками.

## Как чинить ошибку вместе с ИИ

1. Скопируй трейсбек целиком — от «Traceback» до последней строки. Бот из урока 6 сам пришлёт его тебе в личку.
2. Приложи код хендлера, где упало, и версию aiogram.
3. Напиши, что делал перед ошибкой: какую кнопку нажал, что отправил.
4. Попроси сначала объяснить причину, потом дать исправление. Так ты учишься, а не копируешь.

## Чек-лист: проверь код от ИИ перед запуском

1. Синтаксис aiogram 3: нет `executor`, `message_handler`, `dp.register_message_handler`.
2. Токенов и ID нет в коде — только `.env`.
3. SQL только с `?`, без f-строк.
4. В каждом callback-хендлере есть `callback.answer()`.
5. Порядок хендлеров: fallback последний, /cancel выше состояний.
6. Нет блокирующих вызовов в async: `requests` и `time.sleep` заменены на `aiohttp / httpx` и `asyncio.sleep`.
7. Текст пользователя не уходит в `parse_mode="HTML"` без `html.escape()`.
8. У админских команд и кнопок есть фильтр по `OWNER_ID`.

9. Фоновые задачи обернуты в try/except, ссылка на задачу сохранена.
10. Ошибки не глотаются молча: есть logging и обработчик ошибок.

## Приложение Б. Частые ошибки и как их лечить

---

- **TelegramConflictError: terminated by other getUpdates request** — с этим токеном запущены два бота. Останови лишний.
- **TelegramUnauthorizedError** — неверный или отозванный токен. Возьми новый у BotFather.
- **Bad Request: chat not found** — неверный CHAT\_ID (у групп он начинается с -100) или бота нет в чате.
- **Forbidden: bot can't initiate conversation with a user** — человек не запускал бота. Писать первым бот может только тем, кто нажал «Запустить», или в течение 5 минут после заявки в чат.
- **Forbidden: bot was blocked by the user** — человек заблокировал бота. Отметь в базе и не пиши ему.
- **Bad Request: can't parse entities** — включён `parse_mode`, а в тексте неэкранированные «<», «>» или «&».
- **Bad Request: message is not modified** — `edit_text` с тем же текстом и кнопками. Проверь, изменилось ли что-то, перед редактированием.
- **Bad Request: query is too old** — на callback ответили позже чем через 15 секунд или после перезапуска. Отвечай `callback.answer()` в начале долгих хендлеров.
- **Bad Request: BUTTON\_DATA\_INVALID** — `callback_data` длиннее 64 байт. Кириллица весит 2 байта на букву.
- **Bad Request: HIDE\_REQUESTER\_MISSING** — заявку уже приняли, отклонили или она истекла.
- **Кнопка крутится и ничего не происходит** — нет `callback.answer()` или хендлер не нашёлся: проверь фильтр `CallbackData`.
- **Бот молчит в группе** — в группах бот по умолчанию видит только команды. Сделай его админом или выключи `privacy mode` у BotFather (`/setprivacy`).
- **Хендлер не срабатывает** — его перехватил хендлер выше. Подними нужный выше общих.
- **Заявки не приходят** — у бота нет права приглашать пользователей или ссылка создана без «Заявок на вступление».

## Приложение В. Полный код bot.py

---

Сверься с ним свой файл, если что-то не работает. Вместо «ключ1»...«ключ7» — твои ключи.

```
import asyncio
import hashlib
import logging
import os
import time
import traceback
from contextlib import suppress
from datetime import timedelta

import aiosqlite
from aiogram import BaseMiddleware, Bot, Dispatcher, F, Router
from aiogram.exceptions import TelegramAPIError, TelegramBadRequest
from aiogram.filters import Command, CommandObject, CommandStart
from aiogram.filters.callback_data import CallbackData
from aiogram.fsm.context import FSMContext
from aiogram.fsm.state import State, StatesGroup
from aiogram.types import (CallbackQuery, ChatJoinRequest, ErrorEvent, LabeledPrice,
                           Message, PreCheckoutQuery)
from aiogram.utils.deep_linking import create_start_link
from aiogram.utils.keyboard import InlineKeyboardBuilder
from dotenv import load_dotenv

load_dotenv()
OWNER_ID = int(os.getenv("OWNER_ID", "0"))
CHAT_ID = int(os.getenv("CHAT_ID", "0"))
DB = "kluchnik.db"
PRICE_STARS = 100
SUB_DAYS = 30
KEYS = ["ключ1", "ключ2", "ключ3", "ключ4", "ключ5", "ключ6", "ключ7"]

router = Router()

# ----- упок 4: база -----
async def init_db():
    async with aiosqlite.connect(DB) as db:
        await db.executescript(
            """
            CREATE TABLE IF NOT EXISTS users (
                user_id INTEGER PRIMARY KEY,
                source TEXT,
                joined TEXT DEFAULT CURRENT_TIMESTAMP
            );
            CREATE TABLE IF NOT EXISTS subs (
                user_id INTEGER PRIMARY KEY,
                until TEXT NOT NULL,
                reminded INTEGER DEFAULT 0
            );
            """
        )
    await db.commit()
```

```

async def save_user(user_id: int, source: str | None):
    async with aiosqlite.connect(DB) as db:
        await db.execute(
            "INSERT OR IGNORE INTO users (user_id, source) VALUES (?, ?)",
            (user_id, source),
        )
        await db.commit()

async def sources_report():
    async with aiosqlite.connect(DB) as db:
        cur = await db.execute(
            "SELECT COALESCE(source, 'без метки'), COUNT(*) AS c FROM users "
            "GROUP BY source ORDER BY c DESC"
        )
        return await cur.fetchall()

# ----- урок 5: подписки -----
async def extend_sub(user_id: int, days: int) -> str:
    period = f"+{days} days"
    async with aiosqlite.connect(DB) as db:
        await db.execute(
            "INSERT INTO subs (user_id, until) VALUES (?, datetime('now', ?)) "
            "ON CONFLICT(user_id) DO UPDATE SET "
            "until = datetime(max(until, datetime('now')), ?), reminded = 0",
            (user_id, period, period),
        )
        await db.commit()
        cur = await db.execute("SELECT until FROM subs WHERE user_id = ?", (user_id,))
        return (await cur.fetchone())[0]

async def sub_until(user_id: int) -> str | None:
    async with aiosqlite.connect(DB) as db:
        cur = await db.execute(
            "SELECT until FROM subs WHERE user_id = ? AND until > datetime('now')",
            (user_id,),
        )
        row = await cur.fetchone()
        return row[0] if row else None

async def check_subs(bot: Bot):
    async with aiosqlite.connect(DB) as db:
        cur = await db.execute(
            "SELECT user_id, until FROM subs WHERE reminded = 0 "
            "AND until > datetime('now') AND until <= datetime('now', '+3 days')"
        )
        for user_id, until in await cur.fetchall():
            with suppress(TelegramAPIError):
                await bot.send_message(user_id, f"Доступ заканчивается {until} UTC. Продлить:
/buy")
            await db.execute("UPDATE subs SET reminded = 1 WHERE user_id = ?", (user_id,))

        cur = await db.execute("SELECT user_id FROM subs WHERE until <= datetime('now')")
        for (user_id,) in await cur.fetchall():

```

```

        try:
            await bot.ban_chat_member(CHAT_ID, user_id)
            await bot.unban_chat_member(CHAT_ID, user_id, only_if_banned=True)
        except TelegramBadRequest as e:
            logging.warning("Не смог удалить %s: %s", user_id, e)
            await db.execute("DELETE FROM subs WHERE user_id = ?", (user_id,))
        await db.commit()

async def watch_subs(bot: Bot):
    while True:
        try:
            await check_subs(bot)
        except Exception:
            logging.exception("Проверка подписок упала")
        await asyncio.sleep(600)

# ----- урок 2: заявки -----
class Gate(CallbackData, prefix="jr"):
    act: str
    user_id: int

# ----- урок 3: анкета -----
class Apply(StatesGroup):
    waiting_who = State()
    waiting_from = State()

# ----- урок 6: надёжность -----
class AntiFlood(BaseMiddleware):
    def __init__(self, delay: float = 0.7):
        self.delay = delay
        self.last: dict[int, float] = {}

    async def __call__(self, handler, event, data):
        user = data.get("event_from_user")
        if user:
            now = time.monotonic()
            if now - self.last.get(user.id, 0) < self.delay:
                return # слишком часто: молча пропускаем
            self.last[user.id] = now
        return await handler(event, data)

router.message.outer_middleware(AntiFlood())

# ----- урок 8: пропуск -----
def make_proof(user_id: int, bot_id: int, word: str) -> str:
    raw = "|".join([*KEYS, str(user_id), str(bot_id), word])
    return hashlib.sha256(raw.encode()).hexdigest()[:10].upper()

# ----- команды: порядок важен -----
@router.message(CommandStart(deep_link=True, deep_link_encoded=True))
async def start_from_link(message: Message, command: CommandObject):

```

```

    await save_user(message.from_user.id, command.args)
    await message.answer(f"Привет! Ты пришёл по метке: {command.args}")

@router.message(CommandStart())
async def start(message: Message):
    await save_user(message.from_user.id, None)
    await message.answer("Привет! Я Ключник, бот закрытого чата.\n/buy – купить доступ\n/status – мой доступ")

@router.message(Command("id"))
async def my_id(message: Message):
    await message.answer(f"Твой ID: {message.from_user.id}")

@router.message(Command("chatid"))
async def chat_id(message: Message):
    await message.answer(f"ID этого чата: {message.chat.id}")

@router.message(Command("link"), F.from_user.id == OWNER_ID)
async def make_link(message: Message, command: CommandObject, bot: Bot):
    if not command.args:
        await message.answer("Формат: /link метка, например /link youtube")
        return
    await message.answer(await create_start_link(bot, command.args, encode=True))

@router.message(Command("cancel"))
async def cancel(message: Message, state: FSMContext):
    await state.clear()
    await message.answer("Отменил.")

@router.message(Command("buy"))
async def buy(message: Message):
    await message.answer_invoice(
        title="Доступ в закрытый чат",
        description=f"{SUB_DAYS} дней доступа",
        payload=f"sub:{SUB_DAYS}",
        currency="XTR",
        prices=[LabeledPrice(label="Доступ", amount=PRICE_STARS)],
    )

@router.message(Command("status"))
async def status(message: Message):
    until = await sub_until(message.from_user.id)
    await message.answer(f"Доступ до {until} UTC." if until else "Доступа нет. Купить: /buy")

@router.message(Command("admin"), F.from_user.id == OWNER_ID)
async def admin(message: Message):
    rows = await sources_report()
    lines = [f"{source}: {count}" for source, count in rows] or ["пока никого"]
    await message.answer("Откуда пришли:\n" + "\n".join(lines))

```

```

@router.message(Command("refund"), F.from_user.id == OWNER_ID)
async def refund(message: Message, command: CommandObject, bot: Bot):
    parts = (command.args or "").split()
    if len(parts) != 2 or not parts[0].isdigit():
        await message.answer("Формат: /refund USER_ID CHARGE_ID")
        return
    await bot.refund_star_payment(int(parts[0]), parts[1])
    await message.answer("Звёзды возвращены.")

@router.message(Command("proof"), F.from_user.id == OWNER_ID)
async def proof(message: Message, command: CommandObject, bot: Bot):
    if not command.args:
        await message.answer("Формат: /proof СЛОВО_ОТ_ПРИВРАТНИКА")
        return
    word = command.args.strip()
    code = make_proof(message.from_user.id, bot.id, word)
    await message.answer(f"PROOF {word} {code}")

# ----- заявки и анкета -----
@router.chat_join_request(F.chat.id == CHAT_ID)
async def on_join_request(request: ChatJoinRequest, bot: Bot):
    kb = InlineKeyboardBuilder()
    kb.button(text="Я человек", callback_data=Gate(act="human", user_id=request.from_user.id))
    await bot.send_message(
        request.user_chat_id,
        f"Ты подал заявку в «{request.chat.title}». Нажми кнопку, чтобы продолжить.",
        reply_markup=kb.as_markup(),
    )

@router.callback_query(Gate.filter(F.act == "human"))
async def human(callback: CallbackQuery, state: FSMContext):
    await state.set_state(Apply.waiting_who)
    await callback.message.edit_text("Спасибо! Два вопроса.\n1. Кто ты и чем занимаешься?")
    await callback.answer()

@router.callback_query(Gate.filter(F.act.in_({"ok", "no"})), F.from_user.id == OWNER_ID)
async def decide(callback: CallbackQuery, callback_data: Gate, bot: Bot):
    uid, ok = callback_data.user_id, callback_data.act == "ok"
    try:
        if ok:
            await bot.approve_chat_join_request(CHAT_ID, uid)
        else:
            await bot.decline_chat_join_request(CHAT_ID, uid)
    except TelegramBadRequest as e:
        await callback.answer(f"Не вышло: {e.message}"[:200], show_alert=True)
        return
    with suppress(TelegramAPIError):
        await bot.send_message(uid, "Заявка принята. Добро пожаловать!" if ok else "Заявка отклонена.")
    verdict = "принята" if ok else "отклонена"
    await callback.message.edit_text(f"{callback.message.text}\n\nЗаявка {verdict}.")
    await callback.answer()

```

```

@router.message(Apply.waiting_who, F.text)
async def apply_who(message: Message, state: FSMContext):
    await state.update_data(who=message.text)
    await state.set_state(Apply.waiting_from)
    await message.answer("2. Откуда узнал о нас?")

@router.message(Apply.waiting_from, F.text)
async def apply_from(message: Message, state: FSMContext, bot: Bot):
    data = await state.get_data()
    await state.clear()
    user = message.from_user
    nick = f" @{user.username}" if user.username else ""
    kb = InlineKeyboardBuilder()
    kb.button(text="Принять", callback_data=Gate(act="ok", user_id=user.id))
    kb.button(text="Отклонить", callback_data=Gate(act="no", user_id=user.id))
    await bot.send_message(
        OWNER_ID,
        f"Заявка: {user.full_name}{nick}, id {user.id}\n\n"
        f"Кто: {data['who']}\nОткуда: {message.text}",
        reply_markup=kb.as_markup(),
    )
    await message.answer("Анкета у админа. Ответ придёт сюда.")

# ----- оплата -----
@router.pre_checkout_query()
async def pre_checkout(query: PreCheckoutQuery):
    await query.answer(ok=True)

@router.message(F.successful_payment)
async def paid(message: Message, bot: Bot):
    pay = message.successful_payment
    until = await extend_sub(message.from_user.id, SUB_DAYS)
    link = await bot.create_chat_invite_link(CHAT_ID, member_limit=1,
    expire_date=timedelta(days=1))
    await message.answer(f"Оплата прошла. Доступ до {until} UTC.\nТвоя одноразовая ссылка:
    {link.invite_link}")
    await bot.send_message(
        OWNER_ID,
        f"+{pay.total_amount} звёзд от {message.from_user.id}\n"
        f"Возврат: /refund {message.from_user.id} {pay.telegram_payment_charge_id}",
    )

# ----- ошибки и всё остальное -----
@router.errors()
async def on_error(event: ErrorEvent, bot: Bot):
    logging.error("Ошибка: %r", event.exception, exc_info=event.exception)
    tb = "".join(traceback.format_exception(event.exception))
    with suppress(TelegramAPIError):
        await bot.send_message(OWNER_ID, f"Ошибка в боте: {event.exception!r}\n\n{tb[-3500:]}")

@router.message(F.chat.type == "private")
async def fallback(message: Message):

```

```
await message.answer("Не понял. Команды: /buy, /status")

async def main():
    logging.basicConfig(level=logging.INFO)
    await init_db()
    bot = Bot(os.environ["BOT_TOKEN"])
    dp = Dispatcher()
    dp.include_router(router)
    watcher = asyncio.create_task(watch_subs(bot)) # держим ссылку, иначе задачу съест сборщик
    мусора
    await dp.start_polling(bot)
    watcher.cancel()

if __name__ == "__main__":
    asyncio.run(main())
```

Ты прошёл путь от токена у BotFather до бота, который сторожит чат, принимает деньги и сам живёт на сервере. Документация aiogram: [docs.aiogram.dev](https://docs.aiogram.dev).